

面向 LLM / Agent Systems 科研的深度学习零基础课

第 09 讲

Activation Function: ReLU、Sigmoid、Tanh、GELU 与 SiLU

本讲目标：理解激活函数为什么是“非线性”的来源；能逐个数字手算常见激活函数，读懂曲线与 Shape，并连接 Transformer FFN / MLP。

讲义完整版

本课程采用 Just-in-Time Math：数学只讲到足以理解深度学习，并通过手算 + NumPy + 后续 PyTorch 建立底层直觉。

目录

1	这一讲解决什么问题？	3
1.1	最低验收标准	3
2	【原理】激活函数到底是什么？	3
3	【通俗解释】像给每个数字装一个“非线性闸门”	4
4	ReLU：最容易理解的激活函数	4
4.1	定义	4
4.2	为什么它能产生非线性？	5
5	Sigmoid：把数字压到 0 到 1	5
5.1	必要公式	5
5.2	为什么常被理解成“门”？	6
5.3	什么是饱和？	6
6	Tanh：把数字压到 -1 到 1	6
7	GELU：从“硬切断”变成平滑调节	7
8	SiLU：输入乘上自己的 Sigmoid 门	7
9	【图解】五种激活函数长什么样？	8
10	五种函数到底有什么区别？	9
11	【Shape】Activation 通常不改 Shape	9
12	【代码实验】先手写，再用框架验证	10
13	【实验结果】加入非线性后为什么不能再折叠？	11
14	【科研视角】论文改变 Activation 时到底在测什么？	11
15	【常见误区】	12
16	【与 Transformer / LLM / Agent Systems 的关系】	12
17	本讲知识链	13

18 课后检查	13
19 下一讲预告	14

1 这一讲解决什么问题？

上一讲已经证明：如果网络只是连续堆叠 Linear/Affine，哪怕堆很多层，最终仍可折叠为一个 Linear/Affine。

因此，神经网络若想表示更复杂的关系，必须在 Linear 之间插入一种不能被矩阵乘法吸收掉的操作，这就是 **Activation Function（激活函数）**。

本讲核心结论

一个典型神经网络层不是只有：

$$Z = XW + b$$

而是：

$$A = \phi(Z)$$

其中 ϕ 就是激活函数。它通常**逐元素**作用：输入 Shape 不变，但每个数字被非线性地重新映射。

与 LLM / Transformer / Agent Systems 的连接

你以前在 LLM 课程中已经见过 Transformer 的 FFN / MLP：

$$\text{Linear} \rightarrow \text{GELU/SiLU} \rightarrow \text{Linear}$$

今天不再把 GELU、SiLU 当作一个方框，而是从数字层面拆开它们。

1.1 最低验收标准

学完后你应该能：

1. 解释为什么没有 Activation 的多层 Linear 仍然不够；
2. 手算 ReLU，理解 Sigmoid/Tanh/GELU/SiLU 的输入输出范围；
3. 知道激活函数通常逐元素工作，不改变 Shape；
4. 解释 ReLU、GELU、SiLU 在负数区域行为为什么不同；
5. 解释 Sigmoid/Tanh 的饱和直觉；
6. 读懂 Linear -> Activation -> Linear；
7. 连接 Transformer FFN 与现代 LLM 中常见的 GELU / SiLU / SwiGLU。

2 【原理】激活函数到底是什么？

设 Linear 层先产生：

$$Z = XW + b$$

Z 叫 **pre-activation (激活前值)**。随后将每一个 z 送进一个函数：

$$a = \phi(z)$$

得到 **activation (激活值)**。

若 Z 是矩阵：

$$Z = \begin{bmatrix} -2 & 1 \\ 3 & -4 \end{bmatrix}$$

以 ReLU 为例，逐元素得到：

$$\text{ReLU}(Z) = \begin{bmatrix} 0 & 1 \\ 3 & 0 \end{bmatrix}$$

逐元素是关键

激活函数一般不是把整个矩阵乘起来，也不是混合不同 Token。它通常对每一个元素单独做同一种函数运算。因此：

$$(B, T, D) \rightarrow (B, T, D)$$

Shape 通常保持不变。

3 【通俗解释】像给每个数字装一个“非线性闸门”

Linear 做的是加权混合：哪些输入应该加强，哪些应该削弱，如何重新组合。

Activation 做的是：**组合完以后，这个数字应该怎样通过？**

可以把它想成一个“闸门规则”：

- ReLU：负数全部关掉，正数原样通过；
- Sigmoid：把任何数压到 0 和 1 之间；
- Tanh：把任何数压到 -1 和 1 之间；
- GELU / SiLU：不是硬切断，而是平滑地控制通过程度。

注意：这是帮助理解的比喻。数学上它们只是确定的函数。

4 ReLU：最容易理解的激活函数

4.1 定义

ReLU

$$\text{ReLU}(x) = \max(0, x)$$

逐个数字：

$$\text{ReLU}(-3) = 0$$

$$\text{ReLU}(0) = 0$$

$$\text{ReLU}(2) = 2$$

具体数字

输入：

$$[-3, -1, 0, 1, 3]$$

输出：

$$[0, 0, 0, 1, 3]$$

负数被截断为 0，正数保持原值。

4.2 为什么它能产生非线性？

因为 ReLU 在 $x = 0$ 左右采用两套规则：

$$x < 0 \Rightarrow 0, \quad x > 0 \Rightarrow x$$

因此整个函数有一个“折点”。这种折点无法被一个全局的 $ax + b$ 精确表示。

常见误区

“ReLU 很简单，所以能力弱”是错误理解。网络可以通过大量 Linear + ReLU 组合出非常复杂的分段函数。复杂性来自许多简单非线性单元的组合。

5 Sigmoid：把数字压到 0 到 1

5.1 必要公式

Sigmoid

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

这里的 e 是自然常数，约等于 2.718。指数与概率将在后面的 Softmax / Cross Entropy 阶段进一步补数学。

几个关键点：

$$\sigma(0) = 0.5$$

当 x 很大时：

$$\sigma(x) \rightarrow 1$$

当 x 很小时:

$$\sigma(x) \rightarrow 0$$

实验中的结果:

$$\sigma(-3) \approx 0.0474, \quad \sigma(3) \approx 0.9526$$

5.2 为什么常被理解成“门”?

因为输出总在 $(0, 1)$ 。例如 0.02 像“几乎关闭”，0.95 像“几乎打开”。

这也是 LSTM 等门控结构常使用 Sigmoid 的原因之一。

5.3 什么是饱和?

当输入非常大或非常小时，Sigmoid 曲线已经很平：再改变一点输入，输出变化很小。这种区域叫 **saturation (饱和)**。

后面学习 Gradient 时你会看到：很平的区域意味着梯度可能变小，从而让训练变慢。

6 Tanh：把数字压到 -1 到 1

Tanh 的范围

$$-1 < \tanh(x) < 1$$

而且:

$$\tanh(0) = 0$$

实验中的结果:

$$\tanh(-1) \approx -0.7616$$

$$\tanh(1) \approx 0.7616$$

与 Sigmoid 的视觉差别

Sigmoid 输出围绕 0.5；Tanh 输出围绕 0。Tanh 可以自然表达正负两种方向，但它在绝对值很大的区域同样会饱和。

与 LLM / Transformer / Agent Systems 的连接

RNN / LSTM 时代，Tanh 和 Sigmoid 很常见。之后你学习 RNN / LSTM 时会再次看到：Sigmoid 适合门控比例，Tanh 常用于产生有正有负的候选状态。

7 GELU：从“硬切断”变成平滑调节

GELU 全称：**Gaussian Error Linear Unit**。

一个常见的概念公式是：

GELU 的核心形式

$$\text{GELU}(x) = x\Phi(x)$$

$\Phi(x)$ 是标准正态分布的累积分布函数。当前不需要学习概率论细节，只要理解： $\Phi(x)$ 会产生一个 0 到 1 之间、随 x 平滑变化的权重。

因此 GELU 不像 ReLU 那样：

负数 $\Rightarrow$ 全部变 0

而是允许一些较小的负值以较弱形式通过。

实验中：

$$\text{GELU}(-1) \approx -0.1588$$

$$\text{GELU}(1) \approx 0.8412$$

与 LLM / Transformer / Agent Systems 的连接

GPT-2、BERT 等 Transformer 架构中广泛使用 GELU。你以前看到的“Transformer MLP 里的 GELU”就是本节这个逐元素函数。

8 SiLU：输入乘上自己的 Sigmoid 门

SiLU 全称：**Sigmoid Linear Unit**，也常被称为 Swish（特定参数形式）。

SiLU

$$\text{SiLU}(x) = x\sigma(x)$$

逐项拆开：

1. 先计算 $\sigma(x)$ ，得到 0 到 1 之间的门值；
2. 再用 x 乘这个门值；
3. 正的大输入大部分通过；
4. 小的或负的输出被平滑削弱，而非全部硬切成 0。

实验中：

$$\text{SiLU}(-1) \approx -0.2689$$

$$\text{SiLU}(1) \approx 0.7311$$

与 LLM / Transformer / Agent Systems 的连接

很多现代 decoder-only LLM 的 FFN 使用 SiLU 作为门控结构的一部分。你常见的 **SwiGLU** 不是“另一个单独的激活函数名”那么简单，而是把 SiLU 与 gating（门控）结合起来的 FFN 结构。后面重建 Transformer 时再完整拆开。

9 【图解】五种激活函数长什么样？

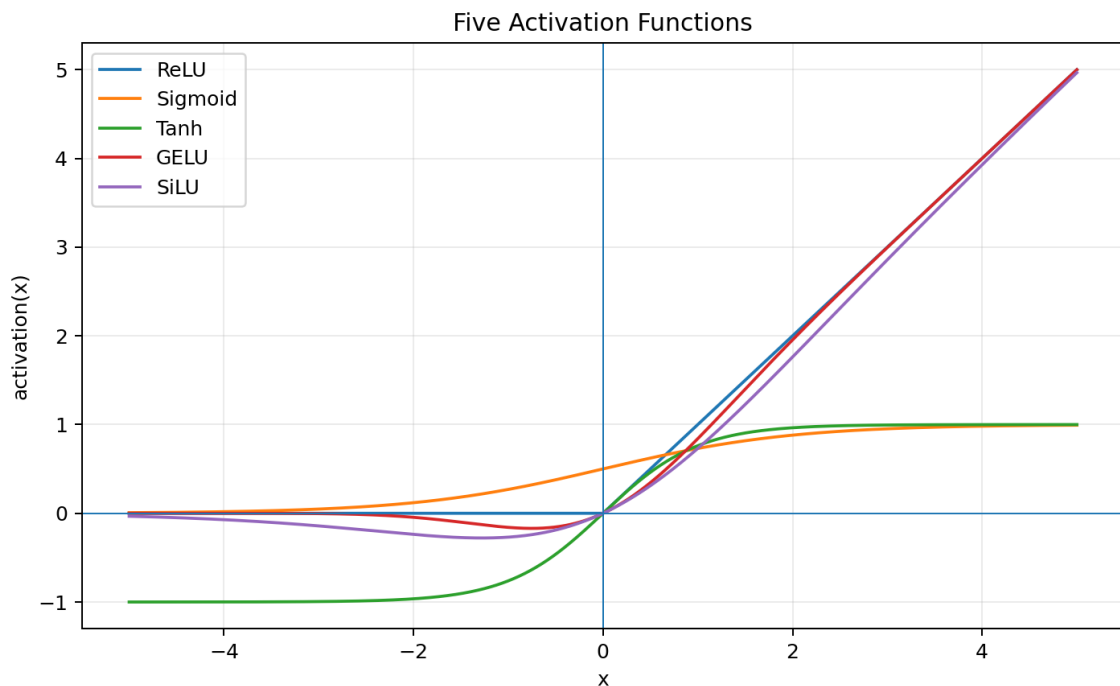


图 1: ReLU、Sigmoid、Tanh、GELU、SiLU 曲线对照。横轴是输入，纵轴是输出。

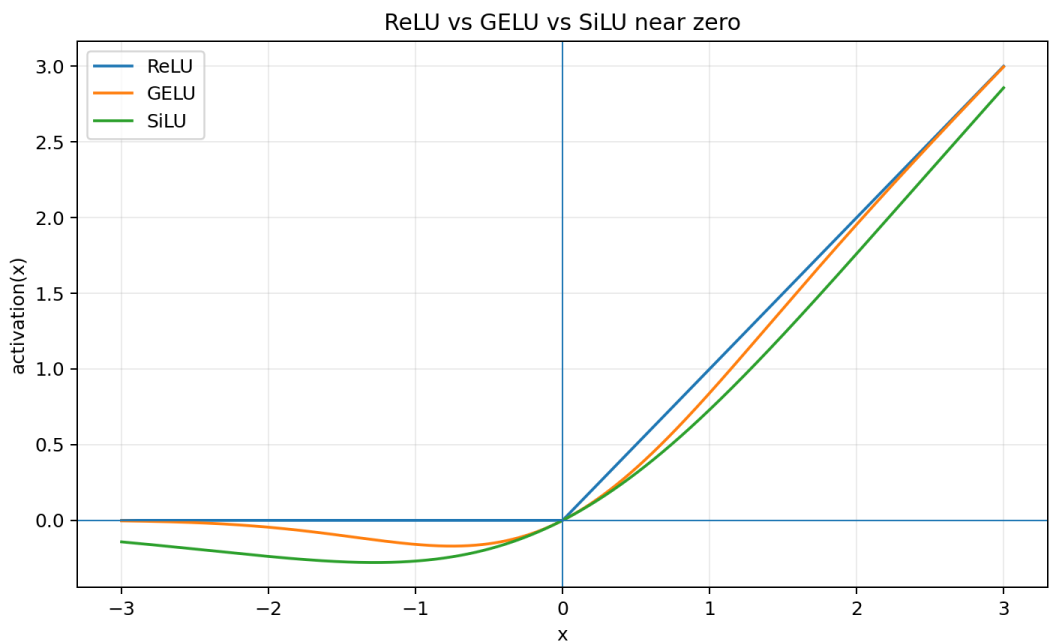


图 2: 放大 0 附近：ReLU 是硬折点；GELU 与 SiLU 更平滑，并允许部分负值通过。

10 五种函数到底有什么区别？

函数	输出范围	负数行为	核心直觉
ReLU	$[0, \infty)$	负数直接变 0	简单、硬截断、分段线性
Sigmoid	$(0, 1)$	压到接近 0	像 0–1 门控；大绝对值区域会饱和
Tanh	$(-1, 1)$	保留负号并压缩	以 0 为中心；同样会饱和
GELU	无固定窄范围	平滑保留部分负值	平滑“选择”输入，Transformer 常见
SiLU	无固定窄范围	平滑保留部分负值	x 乘自己的 Sigmoid 门；现代 LLM 常见

不要背“谁最好”

激活函数不是脱离架构、初始化、Normalization、优化器就能单独排名的组件。科研论文若替换 Activation，应该在尽可能相同的其余条件下比较，而不是看到一个模型更好就把全部提升归因于激活函数。

11 【Shape】Activation 通常不改 Shape

设：

$$X : (B, D_{in})$$

$$W : (D_{in}, D_{out})$$

则：

$$Z = XW + b : (B, D_{out})$$

再应用 ReLU/GELU/SiLU：

$$A = \phi(Z) : (B, D_{out})$$

Shape 不变。

在 Transformer 中：

$$Z : (B, T, D_{ff})$$

经过 GELU/SiLU：

$$A : (B, T, D_{ff})$$

仍然是相同 Shape。

一个很实用的读代码习惯

看到 Activation 时，先问：

1. 它是不是逐元素？
2. 如果是，Shape 应该是否保持？
3. 它改变的是维度，还是数值分布？

对 ReLU/GELU/SiLU 等常见激活，答案通常是：**Shape 保持，数值变化。**

12 【代码实验】先手写，再用框架验证

NumPy 中：

```
import numpy as np

def relu(x):
    return np.maximum(0.0, x)

def sigmoid(x):
    return 1.0 / (1.0 + np.exp(-x))

def silu(x):
    return x * sigmoid(x)

x = np.array([-3., -1., 0., 1., 3.])
print(relu(x))
print(sigmoid(x))
print(silu(x))
```

PyTorch 的作用不是改变数学，而是提供经过优化的实现：

```
import torch.nn.functional as F

A_relu = F.relu(Z)
A_gelu = F.gelu(Z)
A_silu = F.silu(Z)
```

本讲实验中 NumPy 与 PyTorch 的最大差异只有浮点误差量级。

13 【实验结果】加入非线性后为什么不能再折叠？

考虑：

$$y = 2 \cdot \text{ReLU}(3x - 1) + 0.5$$

由于 ReLU 在某一点发生规则切换，整个函数出现折点。

我们让程序寻找“最佳单一直线”去拟合它，仍出现明显误差：

$$\text{max absolute error} \approx 3.359$$

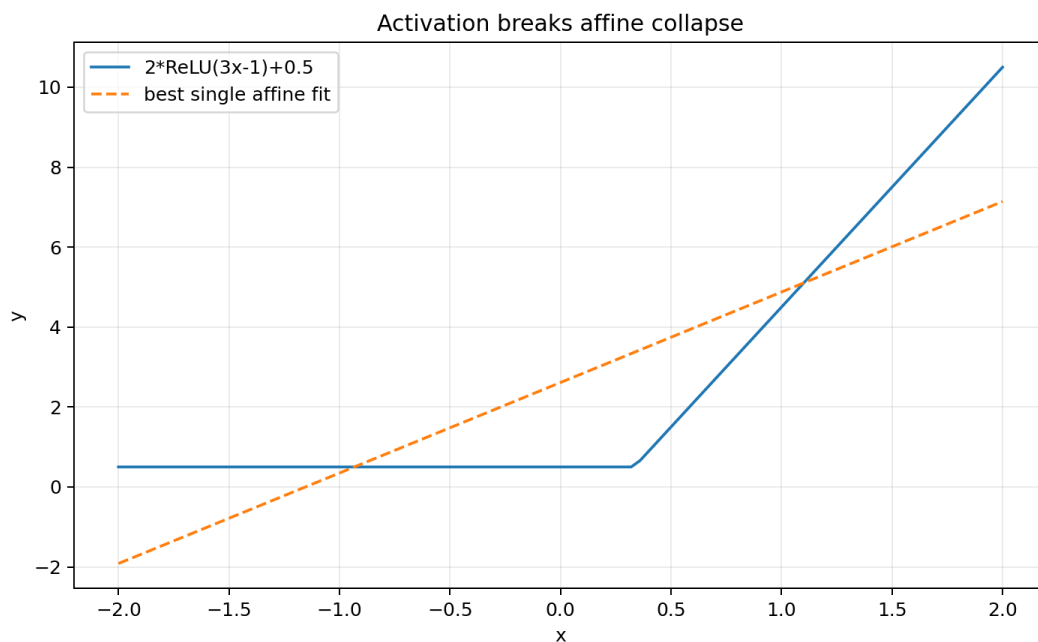


图 3: 非线性网络与最佳单一 affine 拟合。虚线无法在全范围内精确重合。

这就是上一讲“为什么仅有 Linear 不够”的实验闭环。

14 【科研视角】论文改变 Activation 时到底在测什么？

如果一篇论文做：

$$\text{GELU} \rightarrow \text{SiLU}$$

或：

普通 FFN $\rightarrow$ SwiGLU FFN

它可能改变：

- 每层输出的数值分布；
- 负值如何通过；
- 梯度传播特性；
- 参数量（若结构同时改变）；
- 训练稳定性和最终性能。

科研视角

做 Ablation 时必须问：到底只改了 Activation，还是连 FFN hidden dimension、参数量、训练预算也一起改了？若多件事同时变化，就不能把结果简单归因于一个组件。

15 【常见误区】

误区 1：激活函数是在“判断神经元是否开启”

这是历史名称带来的直觉。现代网络里，GELU/SiLU 等输出可以是连续的正负实数，并不只是 0/1 开关。

误区 2：Activation 会改变 Tensor 的维度

常见逐元素激活一般不改 Shape。真正改变最后一维的通常是 Linear/Projection 等映射。

误区 3：Sigmoid 输出可以直接当“真实概率”

Sigmoid 的数值在 0–1，不代表只要用了 Sigmoid 就自动获得校准良好的真实概率。概率解释还取决于训练目标、数据和校准。

误区 4：GELU 比 ReLU “先进”，所以任何任务都一定更好

架构选择没有这种无条件结论。GELU/SiLU 在 Transformer 中常见，不等于所有场景都必然优于 ReLU。

16 【与 Transformer / LLM / Agent Systems 的关系】

现在把前九讲连起来：

$$X \xrightarrow{\text{Linear}} Z \xrightarrow{\text{Activation}} A \xrightarrow{\text{Linear}} Y$$

Transformer FFN 可以粗略理解成：

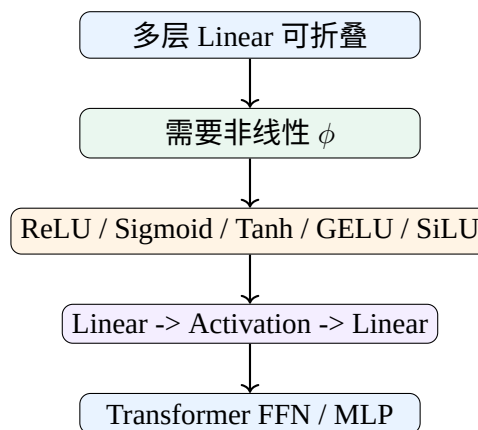
$$(B, T, D) \rightarrow (B, T, D_{ff}) \rightarrow \text{Activation} \rightarrow (B, T, D_{ff}) \rightarrow (B, T, D)$$

其中 Linear 改变特征维度，Activation 改变数值关系但通常不改 Shape。

与 LLM / Transformer / Agent Systems 的连接

以后看 Agent Systems 论文时，你一般不会亲自改 GELU 的公式，但你会依赖底座 LLM。只有理解底座里的 Linear、Activation、Normalization、Attention 等组件，才能判断“模型变化”究竟是系统层变化还是基础模型层变化，也能读懂论文模型配置与训练细节。

17 本讲知识链



18 课后检查

不看资料，尝试回答：

1. 为什么多个 Linear 中间没有 Activation 时仍能折叠？
2. ReLU 对 -2、0、3 的输出分别是什么？
3. Sigmoid 与 Tanh 的输出范围分别是什么？
4. 为什么 GELU / SiLU 与 ReLU 在负数区域不同？
5. 为什么常见激活函数通常不改变 Shape？
6. Transformer FFN 为什么不是简单地 Linear -> Linear？

本节检查点

若你能解释“Linear 负责混合特征，Activation 负责引入非线性；Transformer FFN 把两者组合起来”，并能看懂常见 Activation 的输入输出行为，本讲就达标。

19 下一讲预告

第 10 讲：**MLP / Feed-Forward Neural Network**。

我们会把多个神经元、Linear、Activation 正式组装成一个完整 MLP，并逐层追踪：

Input Shape $\rightarrow$ Hidden Shape $\rightarrow$ Output Shape

同时计算参数量，为后面的 Transformer FFN 奠定结构基础。